Parul® University
Vadodara, Gujarat

NAAC GRADE A++

Information and Communication Technology

Knowledge Representation

Study Guide

Prof. Kusum Sharma
CSE - AIML, PIET
Parul University

Parul® University
Vadodara, Gujarat
NAAC GRADE A++

INDEX

Unit – 3

Knowledge Representation

Knowledge Representation – Representation and Mappings, Different Approaches, Issues in knowledge representation. Predicate Logic - Representation Simple Facts in Logic, Representing Instance and Isa Relationships, Computable Functions and Predicates, Resolution. Propositional Logic: Representation, Inference, Reasoning Patterns, Resolution, First-order Logic: Representation, Inference, Reasoning Patterns, Resolution Representation and Mappings

Introduction

For the purpose of solving complex problems encountered in AI, we need both a large amount of knowledge and some mechanism for manipulating that knowledge to create solutions to new problems. A variety of ways of representing knowledge (facts) have been exploited in AI programs.

In all variety of knowledge representations, we deal with two kinds of entities.

A. Facts: Truths in some relevant world. These are the things we want to represent.

B. Representations of facts in some chosen formalism.

These are things we will actually be able to manipulate. One way to think of structuring these entities is at two levels: (a) the knowledge level, at which facts are described, and (b) the symbol level, at which representations of objects at the knowledge level are defined in terms of symbols that can be manipulated by programs.

The facts and representations are linked with two-way mappings. This link is called representation mappings. The forward representation mapping maps from facts to representations. The backward representation mapping goes the other way, from representations to facts. One common representation is natural language (particularly English) sentences. Regardless of the representation for facts we use in a program, we may also need to be concerned with an English representation of those facts in order to facilitate getting information into and out of the system. We need mapping functions from English sentences to the representation we actually use and from it back to sentences.

Representations and Mappings

- In order to solve complex problems encountered in artificial intelligence, one needs both a large amount of knowledge and some mechanism for manipulating that knowledge to create solutions.

- Knowledge and Representation are two distinct entities. They play central but distinguishable roles in the intelligent system.
- Knowledge is a description of the world. It determines a system's competence by what it knows.
- Moreover, Representation is the way knowledge is encoded. It defines a system's performance in doing something.
- Different types of knowledge require different kinds of representation.

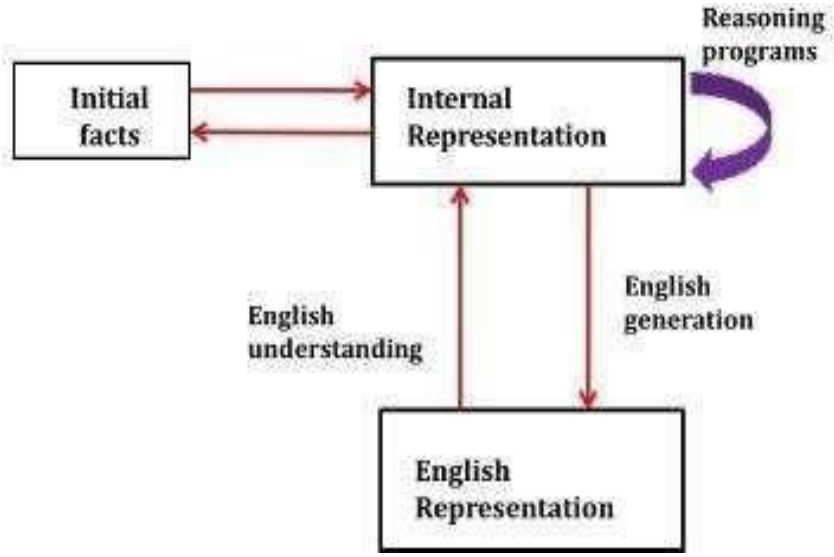


Fig: Mapping between Facts and Representations

The Knowledge Representation models/mechanisms are often based on:

- Logic
- Rules
- Frames
- Semantic Net

Knowledge is categorized into two major types:

1. Tacit corresponds to "informal" or "implicit"
    - Exists within a human being;
    - It is embodied.
    - Difficult to articulate formally.
    - Difficult to communicate or share.
    - Moreover, Hard to steal or copy.

    Drawn from experience, action, subjective insight
2. Explicit formal type of knowledge, Explicit
    - Explicit knowledge
    - Exists outside a human being;
    - It is embedded.

- Can be articulated formally.
- Also, Can be shared, copied, processed and stored.
- So, Easy to steal or copy
- Drawn from the artifact of some type as a principle, procedure, process, concepts. A variety of ways of representing knowledge have been exploited in AI programs. There are two different kinds of entities, we are dealing with.

1. Facts: Truth in some relevant world. Things we want to represent.
2. Also, Representation of facts in some chosen formalism. Things we will actually be able to manipulate.

These entities structured at two levels:

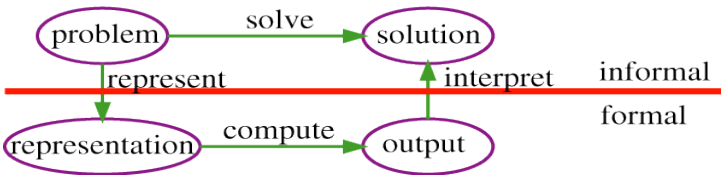
1. The knowledge level, at which facts described.
2. Moreover, The symbol level, at which representation of objects defined in terms of symbols that can manipulate by programs

Representation:

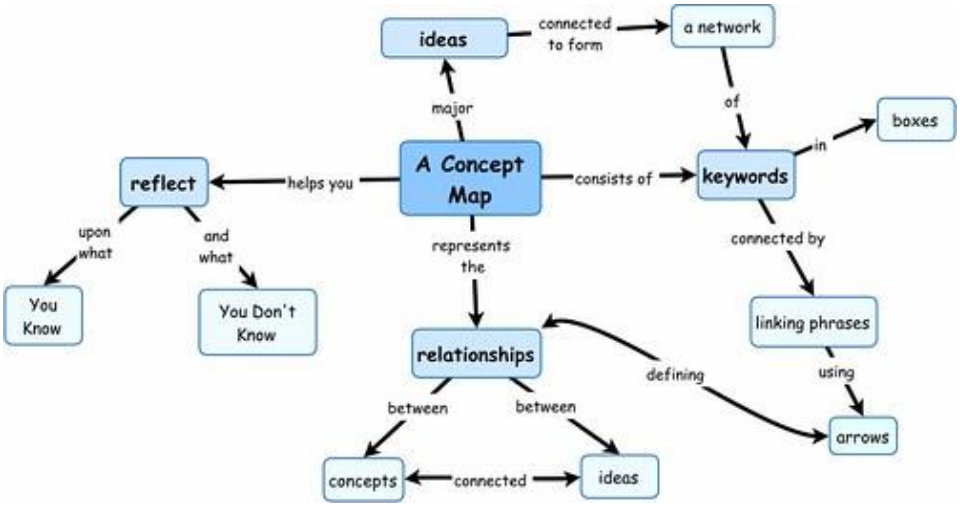
- Definition: Knowledge representation is the field of AI dedicated to representing information about the world in a form that a computer system can utilize to solve complex tasks.

  Artificial intelligence (AI) is based on the core idea of knowledge representation, which tries to capture and organize knowledge in a meaningful and structured fashion. It entails archiving data and making it available to AI systems so they may learn, reason, and make decisions based on knowledge.

- Types:
  - Symbolic: Uses symbols to represent knowledge (e.g., logic, frames).
  - Sub-symbolic: Uses patterns and connections (e.g., neural networks).



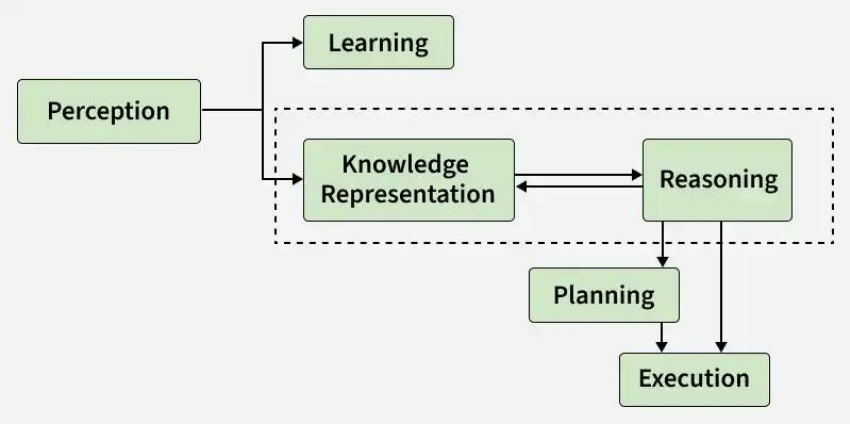
2. Mappings:
   - Mapping: The process of associating one set of data with another. In AI, it involves translating real-world scenarios into a machine-understandable format.
   - Types of mappings:
     - Structural Mapping: Aligning the structure of one domain with another.
     - Semantic Mapping: Aligning meanings across different representations.



Different Approaches

Knowledge Representation (KR) in AI focuses on how machines store and organize real-world information so they can reason, learn, and make intelligent decisions like humans.

- Represents knowledge in a structured form that computers can process.
- Helps AI systems perform reasoning and problem-solving tasks.
- Enables intelligent applications like medical diagnosis and language understanding.
- Allows machines to use stored knowledge and past experiences effectively.



Types of Knowledge

- Declarative Knowledge: Knowledge about facts and concepts it answers what something is . Example: Paris is the capital of France.

- Procedural Knowledge: Knowledge about how to perform a task or solve a problem. Example: Steps to sort numbers using an algorithm.

- Meta knowledge: Knowledge about other knowledge or how knowledge is used. Example: Knowing that a certain rule works better for solving math problems.

- Heuristic Knowledge: Experience based knowledge or rules of thumb used by experts. Example: A doctor using past experience to guess a possible disease.

- Structural Knowledge: Knowledge that shows relationships between concepts. Example: A car is a type of vehicle.

Cycle of Knowledge Representation in AI

The cycle of knowledge representation in AI refers to the iterative process through which AI systems perceive, learn, represent, and apply knowledge to make informed decisions. This cycle is essential for building intelligent systems capable of reasoning and interacting with their environment. Here are the key stages of this cycle:

1. Perception

- Description: The process begins with AI systems perceiving data from their environment. This data could come from sensors, cameras, user inputs, or databases.
- Example: A self-driving car uses sensors to perceive its surroundings, such as detecting pedestrians and other vehicles.

2. Learning

- Description: After perceiving data, the system learns by identifying patterns, relationships, and trends from the data. This can involve supervised learning, unsupervised learning, or reinforcement learning techniques.
- Example: An AI algorithm can learn from historical sales data to predict future trends, helping businesses make data-driven decisions.

3. Knowledge Representation & Reasoning

- Description: In this stage, the system organizes the learned data into structured knowledge, allowing it to reason and draw conclusions. This is where the AI uses knowledge representation techniques to encode the information it has learned.
- Example: A medical diagnosis system represents symptoms, diseases, and treatments as structured knowledge and uses reasoning to suggest the best treatment based on a patient's

symptoms.

4. Planning

- Description: The system uses the represented knowledge to plan actions based on goals and constraints. Planning involves selecting the best course of action based on the available knowledge.
- Example: In robotics, AI systems use planning to determine the most efficient path for a robot to navigate through a room while avoiding obstacles.

5. Execution

- Description: Finally, the system executes the planned actions, completing the cycle. After execution, the AI system can receive feedback and adjust its knowledge and actions accordingly.
- Example: A robot executing the planned actions to pick up and move objects based on the knowledge it has about the environment.

The Relation between Knowledge and Intelligence

Knowledge and intelligence are deeply interconnected in both artificial and human cognition. In AI, intelligence refers to the system's ability to make informed decisions, solve problems, and adapt to new information. Knowledge representation provides the foundation for this intelligence by enabling the AI system to store, organize, and use information effectively.

1. Knowledge as the Building Block of Intelligence

- Just as humans rely on knowledge to make decisions, AI systems require knowledge to exhibit intelligent behavior. Without the ability to represent knowledge, AI systems would be limited to simple, reactive behaviors. Knowledge allows AI to analyze situations, reason about the best course of action, and adapt to new environments.

2. Cognitive Science and AI

- Cognitive science studies how humans represent knowledge and use it to solve problems. AI systems often draw inspiration from cognitive science, simulating human reasoning processes through knowledge representation techniques such as semantic networks, frames, and production rules. This alignment between cognitive science and AI enhances our ability to build intelligent systems that mimic human thought processes.

3. Reasoning and Decision-Making

- The ability to reason—drawing conclusions from known facts and rules—is a hallmark of intelligence in both humans and AI. AI systems that possess knowledge can perform logical

reasoning, make inferences, and predict outcomes. This capability is critical for applications like medical diagnosis, autonomous driving, and financial forecasting.

Approaches to Knowledge Representation

AI systems use different approaches to represent knowledge depending on the nature of the problem and the type of information they need to handle. Here are some key approaches to knowledge representation in AI:

1. Simple Relational Knowledge

- Description: This approach represents knowledge as simple facts in the form of relations between entities. It uses tables or relational databases to store information about objects and their relationships.
- Example: A table in a database could store the relationship between students and their courses, with columns for student names, course names, and grades.
- Strengths: Straightforward and easy to implement, especially in structured environments like databases.
- Weaknesses: Lacks the ability to handle complex relationships or hierarchies.

2. Inheritable Knowledge

- Description: Inheritable knowledge uses hierarchies and inheritance to represent general and specific information about objects. This approach allows entities to inherit properties from higher-level categories.
- Example: In a knowledge base, a "dog" might inherit properties from the more general category "mammal," such as being warm-blooded and having fur.
- Strengths: Efficient in representing hierarchical knowledge and reducing redundancy by reusing information.
- Weaknesses: Can be challenging to represent exceptions or unique cases that don't follow the inheritance structure.

3. Procedural Knowledge

- Description: Procedural knowledge defines sequences of actions or steps needed to accomplish specific tasks. It focuses on "how to" knowledge rather than "what is."
- Example: An AI system for controlling a robot might use procedural knowledge to define the steps required for the robot to pick up an object: locate the object, move towards it, and grip it with an arm.
- Strengths: Useful for automating tasks and guiding AI systems through well-defined procedures.
- Weaknesses: Not suitable for tasks requiring complex reasoning or flexible decision-making.

4. Inferential Knowledge

- Description: This approach involves representing knowledge in a way that allows the AI to infer new information from existing facts and rules. Logical reasoning is applied to draw conclusions.
- Example: Given the facts "All humans are mortal" and "Socrates is a human," an AI system using inferential knowledge can infer that "Socrates is mortal."
- Strengths: Enables AI systems to apply logical reasoning and make deductions.
- Weaknesses: Can be computationally expensive and may struggle with incomplete or uncertain information.

Each of these approaches offers unique benefits and limitations, and they are often used in combination within AI systems to meet the needs of specific tasks or domains

Issues in Knowledge Representation

Several obstacles and potential directions still exist in knowledge representation, despite the substantial progress made in this area. Among the principal difficulties are:

- **Scalability:** Scalability becomes a significant difficulty as knowledge's volume and complexity rise. Large knowledge bases must be efficiently represented and processed using sophisticated methods and distributed computing concepts.
- **Information that is Uncertain or Incomplete:** AI systems frequently work with information that is uncertain or incomplete. A major research area is improving knowledge representation approaches to manage uncertainty and reason with inadequate data.
- **Knowledge Fusion and Integration:** Combining and integrating knowledge from various sources and modalities is a difficult task. The goal of future research is to create methods that make it possible for heterogeneous knowledge to be seamlessly integrated for better AI performance.
- **Explainability & Interpretability:** AI systems should be able to justify their decisions with explanations. Building trust, assuring ethical AI, and satisfying legal standards all depend on the development of clear and understandable knowledge representation approaches.

Knowledge representation techniques

1. Logical Representation

Logical representation uses formal rules and logic to represent knowledge in AI and helps systems draw conclusions based on given conditions. It includes the following ideas:

- Syntax: Rules that decide how symbols and sentences are written in logic.

- Semantics: Rules that give meaning to logical sentences.
- Logical representation mainly includes Propositional Logic and Predicate Logic.

Advantages

- Helps in logical reasoning and decision making.
- Forms the basis of many programming languages.

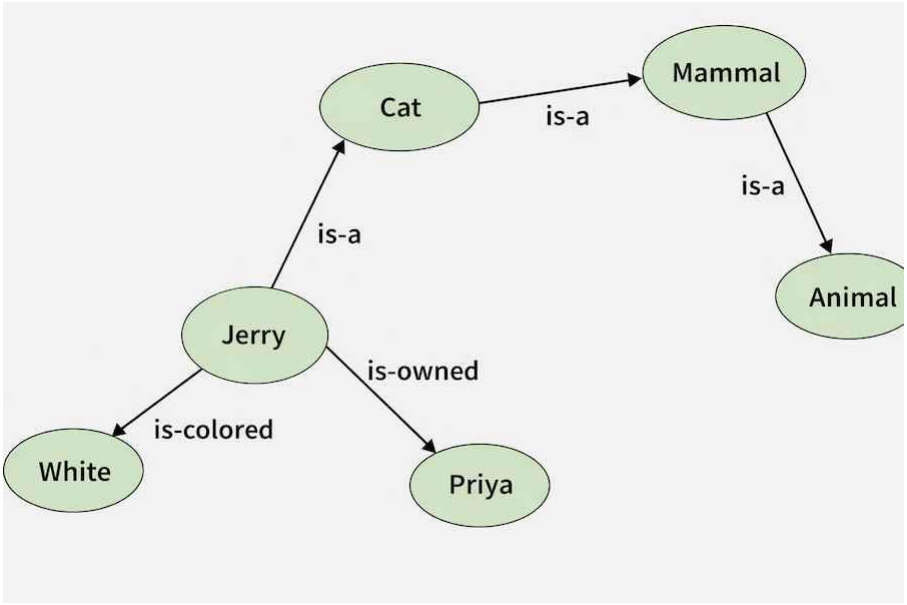
Disadvantages

- Can be difficult to use and understand.
- Reasoning using logic can sometimes be slow or inefficient.

2. Semantic Network Representation

A semantic network represents knowledge using a graph structure with nodes (concepts or objects) and arcs (relationships). It helps show connections between different concepts.

- IS-A relation: Shows inheritance (e.g. Cat is a Mammal).
- Kind-of relation: Shows category relationships.



Semantic networks example

Advantages

- Natural and easy way to represent knowledge.
- Simple to understand and visualize.
- Can be easily expanded.

Disadvantages

- Searching large networks can take more time.
- Difficult to create very large networks like human memory.
- Lacks clear standards for link names and logical expressions.

3. Frame Representation

A frame is a data structure used to represent knowledge about an object or situation using slots (attributes) and values. It organizes related information together and is widely used in NLP and computer vision.

Example: A frame for a Book may include slots such as title, author, year and pages.

Advantages

- Groups related information together, making programming easier.
- Flexible and easy to extend by adding new attributes.
- Easy to understand and visualize.

Disadvantages

- Inference and reasoning are not always efficient.
- Sometimes too generalized for complex reasoning.

4. Production Rules

Production rules represent knowledge in the form of IF–THEN rules. When a condition is satisfied, the system performs the corresponding action. A production rule system has three main parts:

- Set of rules
- Working memory (current problem state)

- Recognize act cycle (process of checking conditions and applying rules)

Example: IF (bus arrives) THEN (get into the bus).

Advantages

- Easy to understand because rules are written in simple language.
- Highly modular; rules can be added, removed, or modified easily.

Disadvantages

- Systems usually do not learn from past results.
- Many active rules can make the system slower or inefficient.

Predicate Logic – Representation

What is Predicate Logic in Artificial Intelligence?

Predicate logic, also known as first-order logic (FOL), is an extension of propositional logic that allows us to express relationships between objects and their properties. In AI, predicate logic is widely used to represent knowledge and perform reasoning in more complex scenarios where relationships matter.

Unlike propositional logic, which deals with simple true/false statements, predicate logic introduces predicates, variables, constants, and quantifiers. These elements help in modeling real-world problems that involve multiple objects and their interactions.

Role of Predicate Logic in AI

- Knowledge Representation: It provides a structure for representing complex facts about objects and their relationships in a system.
- Reasoning: AI systems use predicate logic to infer new information from existing facts, making it suitable for decision-making tasks.

Example:

- “John is the father of Mary” can be represented as: Father(John,Mary)Father(John, Mary)Father(John,Mary)

This expression tells us that there is a relationship (Father) between two objects (John and Mary). Predicate logic enables us to model and reason about such relationships effectively.

Components of Predicate Logic:

Predicate logic involves several key components that allow it to represent relationships and properties of objects in a structured way.

1. Predicates:

- A predicate is a function that returns either true or false based on the relationship between its arguments.
- Example: IsHungry(John)IsHungry(John)IsHungry(John) This predicate represents whether John is hungry, returning true if he is and false if not.

2. Variables:

- Variables are placeholders for objects within a domain. They allow us to represent general statements that apply to multiple objects.
- Example: In IsHungry(x), the variable x can represent any person.

3. Constants:

- Constants represent specific objects or entities in the domain.
- Example: John is a constant in the predicate IsHungry(John).

Structure of Predicates

Predicate consists of two key elements: the predicate symbol and arguments. Predicates are enhanced with quantifiers to specify the scope of variables involved.

1. Predicate Symbol

- The predicate symbol defines the property or relationship being described.
- Example:
  - IsHungry(x) represents whether a person (x) is hungry.
  - Married(x, y) denotes that person x is married to person y.
- Predicates are named based on the relationship or property they represent. The symbol is followed by arguments enclosed in parentheses.

2. Arguments and Arity

- Arguments refer to the specific objects that the predicate is applied to.
- The arity of a predicate refers to the number of arguments it takes.

Examples:

- IsHungry(x): A predicate with 1 argument (arity = 1).
- Married(x, y): A predicate with 2 arguments (arity = 2).
- X(a, b, c): A predicate with 3 arguments (arity = 3), representing something like "a + b + c = 0."

3. Quantifiers in Predicate Logic

Quantifiers allow us to specify the scope of variables. There are two main types:

1. Existential Quantifier (∃)
   - Meaning: There exists at least one object that satisfies the given condition.
   - Example: ∃x IsHungry(x)
     This statement means that at least one person is hungry.
   - Negation: The negation of the existential quantifier means that no such object exists.
     ¬∃x IsHungry(x)
     This means that no one is hungry.
2. Universal Quantifier (∀)
   - Meaning: The given condition holds for all objects in the domain.
   - Example: ∀x (IsHuman(x)→IsMortal(x))
     This means that all humans are mortal.
   - Negation: The negation of the universal quantifier means there is at least one exception.
     ¬∀x IsHuman(x)→IsMortal(x)
     This implies that at least one human is not mortal.

Examples of Predicate Logic

1. Simple Predicate Example:

- Predicate: IsHungry(John)
  - Meaning: This predicate represents the state of whether John is hungry. It takes one argument (John) and returns true if John is hungry, otherwise false.
- Application in AI:
  - In NLP-based chatbots, predicates like this could help infer the user's intent. For example, if a chatbot detects that the user is hungry, it could suggest nearby restaurants.

2. Equality Predicate Example:

- Predicate: E(x,y)≡(x=y)
  - Meaning: This predicate denotes that x is equal to y. It returns true if the two objects are identical.
- Application in AI:

- AI-based reasoning systems use equality predicates to match objects. For example, in a robot warehouse, a robot may use this predicate to determine if an object picked matches the one requested (e.g., E(Package1, RequestedItem)).

3. Mathematical Predicate Example:

- Predicate: X(a,b,c)≡(a+b+c=0)
  - Meaning: This predicate checks whether the sum of a, b, and c equals zero. It returns true if the equation holds, otherwise false.
- Application in AI:
  - In optimization problems, AI models might use mathematical predicates to check if constraints are satisfied. For example, in scheduling systems, such predicates can validate if certain conditions are met (e.g., X(shiftA, shiftB, totalTime) checks if the total shift hours are balanced).

4. Relationship Predicate Example:

- Predicate: M(x,y)≡x is married to y
  - Meaning: This predicate expresses a relationship between two objects, indicating that x is married to y.
- Application in AI:
  - In family tree AI systems, relationship predicates are used to infer relationships among family members. For instance, if M(John, Mary) is true, the system can infer that John is Mary's spouse.

5. Universal Quantification Example:

- Expression: ∀x (IsHuman(x)→IsMortal(x))
  - Meaning: This statement reads as "For all x, if x is human, then x is mortal." It applies to every object in the domain of humans. If an object is found to be human, it must also be mortal for the statement to hold true.
- Application in AI:
  - Knowledge-based systems use such rules to infer properties about objects. For example, in medical diagnosis systems, rules like "All viruses can spread infections" (∀x IsVirus(x) → CanSpreadInfection(x)) help the system reason about diseases.

6. Existential Quantification Example:

- Expression: ∃x IsHungry(x)
  - Meaning: This reads as "There exists at least one x such that x is hungry." It indicates that at least one object in the domain satisfies the condition of being hungry.
- Application in AI:
  - In robot planning, a robot could use existential quantifiers to plan actions. For

example, “There exists a task that requires charging” (∃x TaskRequires(x, Charging)) might guide the robot to prioritize charging tasks.

7. Compound Example with Multiple Quantifiers:

- Expression: ∀x∃y (Parent(x,y))
  - Meaning: This statement means “For every person x, there exists a person y such that x is the parent of y.” It shows how multiple quantifiers can be used together to represent complex relationships.
- Application in AI:
  - This logic is often used in social network AI models to analyze relationships. In a family tree system, the model could use such logic to infer relationships between family members.

Propositions with Multiple Quantifiers

In predicate logic, multiple quantifiers can be used within a single proposition to express more complex ideas. The order of quantifiers is crucial, as it can change the meaning of the statement.

Example 1: Order of Quantifiers Matters

∀x∃y (Parent(x,y))

- Meaning: For every person x, there exists at least one person y such that x is the parent of y.
- Example in AI: In a family tree system, this could represent the rule that every parent must have at least one child.

Now, let’s reverse the quantifiers:

∃y∀x (Parent(x,y))

- Meaning: There exists a person y such that every person x is the parent of y.
- Interpretation: This is logically impossible under normal circumstances, as a single person cannot have all people as parents.

Example 2: Nested Quantifiers in AI

∀x∃y (RobotCanPerform(x,y))

- Meaning: For every task x, there exists a robot y that can perform the task.
- AI Application: This could represent a rule in a robot planning system, where every task must have at least one robot capable of completing it.

Now, consider the reversed version:

$\exists y \forall x$ (RobotCanPerform(x,y))

- Meaning: There exists a robot y that can perform every task x.
- AI Application: This would imply that a single robot can perform all tasks, which may not always be practical.

Impact of Quantifier Order on Meaning

As seen in the examples above, changing the order of quantifiers can completely change the meaning of a proposition. In AI systems, it's essential to use the correct order of quantifiers to ensure that logical statements align with the desired behaviour or knowledge representation.

Knowledge Representation using Predicate Logic in AI

Knowledge representation is a critical part of AI, enabling systems to store, reason, and derive new facts from existing information. Predicate logic offers a structured way to represent complex relationships, facts, and rules in a logical and understandable format.

How Predicate Logic Helps in Knowledge Representation

1. Representing Relationships Between Objects:
    - Example: Married(John,Mary)
    - This expression indicates that John is married to Mary. Predicate logic captures such binary relationships between objects, helping AI systems understand connections.
2. Storing Rules in Knowledge Bases:
    - Rules can be defined using predicates and quantifiers.
    - Example: $\forall x$ (IsHuman(x)$\rightarrow$IsMortal(x)) This rule states that all humans are mortal. AI systems can store these rules and apply them during reasoning.
3. Handling Multiple Entities and Properties:
    - AI systems can store complex information by using predicates with multiple arguments.
    - Example: Parent(John,Mary)$\wedge$Sibling(Mary,Kate) This expresses that John is Mary's parent and Mary is Kate's sibling.
4. Reasoning and Inference:
    - AI systems can use inference rules to derive new facts from stored knowledge.
    - Example: If the system knows that: $\forall x$ (Parent(John,x)$\rightarrow$Loves(John,x)) It can infer that John loves all his children.

Examples of AI Applications Using Predicate Logic

- Natural Language Processing (NLP): NLP systems use predicate logic to understand relationships in text and answer complex queries. For example, in a chatbot, predicate logic

helps infer the user's intent.

- Robot Planning Systems: In a warehouse, robots use predicate logic to plan tasks based on conditions like:
  ∃xTaskRequires(x,Charging)

  This ensures robots prioritize tasks that require charging.
- Expert Systems:
  Predicate logic is used in expert systems (like medical diagnosis tools) to store if-then rules and infer new facts about diseases based on symptoms.

Difference between Predicate Logic and Propositional Logic

Aspect	Predicate Logic	Propositional Logic
Definition	Extends propositional logic by allowing variables and relationships between objects.	Deals with simple statements that are either true or false.
Elements	Uses predicates, variables, constants, and quantifiers.	Uses atomic propositions (simple statements).
Expressiveness	More expressive; can represent relationships and properties.	Less expressive; limited to simple true/false statements.
Example	∀x (IsHuman(x) → IsMortal(x))	It is raining (P).
Quantifiers	Supports existential (∃) and universal (∀) quantifiers.	No quantifiers are used.
Handling of Relationships	Can represent complex relationships, like Parent(John, Mary).	Cannot represent relationships between objects.
Applications	Used in NLP, expert systems, and robot planning.	Used in decision-making systems with simpler conditions.
Complexity	More complex due to the use of variables and quantifiers.	Simpler with fewer components.
Reasoning Capability	Enables inference through rules and relationships.	Limited to evaluating truth values of propositions.

Simple facts in logic

Propositional Logic (PL) is a branch of logic that focuses on statements (propositions) that can be either true or false. It is also known as Boolean logic since the truth values are binary—either True (1) or False (0).

In AI, propositional logic forms the foundation for logical reasoning, allowing systems to represent facts and rules about a problem domain. These rules help the system infer new information or make decisions based on the given inputs.

Propositional logic simplifies knowledge representation by breaking down reasoning into atomic statements or propositions. For example, an AI system used in home automation might have propositions such as:

- P: "The light is on."
- Q: "The window is open."

Using logical connectives, the system can combine these propositions to represent more complex statements like:
"If the light is on and the window is open, turn off the light."

By using propositional logic, AI systems can reason effectively and perform tasks like automated decision-making, knowledge representation, and game playing.

Basic Facts About Propositional Logic

1. Propositions are Declarative Statements:

- In propositional logic, each statement, known as a proposition, is either True or False. Example:
- P: "It is raining." (True or False)
- Q: "The heater is on." (True or False)

2. Atomic Propositions:

- These are simple, indivisible statements that cannot be broken down further. Each atomic proposition represents a basic fact or condition. Example: "The door is closed."

3. Compound Propositions:

- Multiple atomic propositions can be combined using logical connectives (like AND, OR, NOT) to create compound propositions.
  Example: “The door is closed AND the heater is on.”

4. Binary Truth Values:

- Every proposition has a binary truth value: it can only be True (1) or False (0). There are no intermediate states. This simplicity makes propositional logic ideal for clear-cut decisions.

5. Logical Connectives Combine Propositions:

- Logical connectives such as AND, OR, NOT, IF-THEN, and IF AND ONLY IF allow us to create more complex propositions from simple ones.

Representing Instance and Isa Relationships:

Logic allows us to represent two important relationships between objects and categories:

- Instance Relationship: This shows that an object belongs to a specific category.
  - Example: "Tweety is a bird" can be written as: is_a(Tweety, bird). Here, "Tweety" is an instance of the category "bird".
  -
- Isa Relationship ("is-a"): This captures the hierarchical structure of knowledge by representing a subclass relationship between categories.
  - Example: "All birds are animals" can be expressed as: ∀x (bird(x) → animal(x)). ("∀x" is the universal quantifier, meaning "for all x"). This statement says that anything that is a bird is also an animal.

1. Instance Relationships: Specific examples of a general category.
   - Example: Human(Socrates)Human(Socrates)Human(Socrates) represents that Socrates is an instance of the category human.
2. Isa Relationships: Hierarchical relationships between categories.
   - Example: Isa(Dog,Animal)Isa(Dog, Animal)Isa(Dog,Animal) represents that dogs are a subclass of animals.

Computable Functions and Predicates

Predicate logic can represent not just simple relationships but also functions that operate on objects.

- Functions: These take one or more objects as input and produce a specific output object.
    - Example: "father(John)" could be a function that takes a person's name and returns their father's name.
- Computable Predicates: Similar to functions, predicates can take objects as input and evaluate to true or false based on a defined rule.
    - Example: "greater_than(5, 3)" would be a predicate that evaluates to true because 5 is greater than 3.

1. Functions: Mappings from objects to objects.
    - Example: FatherOf(x)FatherOf(x)FatherOf(x) could return the father of xxx.
2. Predicates: Functions that return true or false.
    - Example: GreaterThan(x,y)GreaterThan(x, y)GreaterThan(x,y) returns true if xxx is greater than yyy.

Resolution

Resolution is a powerful technique for automated reasoning in logic. It allows us to derive new logical statements (conclusions) from existing ones (premises). Here's the basic idea:

1. We convert both the premises and the desired conclusion into a specific format called clauses.
2. We apply a set of rules to identify complementary clauses (clauses with opposite literals).
3. By combining and resolving these complementary clauses, we eliminate some symbols and potentially derive a new clause.
4. If this new clause represents a tautology (always true statement), then the desired conclusion can be proven true based on the given premises.

Resolution is a complex topic, but understanding its core idea helps you grasp how logic can be used to reason about knowledge and draw conclusions automatically.

Resolution Rule: A single inference rule used in predicate logic and propositional logic for automated theorem proving.

Process: Convert all statements to a standard form (CNF), then iteratively apply the resolution rule to derive a contradiction.

Propositional Logic

Propositional logic is a simpler form of logic that deals with propositions, which are statements that can be true or false.

1. Representation:

- Propositional Variables: Represented by letters (e.g., P, Q, R). They stand for statements whose truth value we are interested in.
- Logical Connectives: These symbols combine propositions to form more complex statements.
    - AND (∧): Both propositions must be true (e.g., P ∧ Q).
    - OR (∨): At least one proposition must be true (e.g., P ∨ Q).
    - NOT (¬): Negates a proposition (e.g., ¬P).
    - Implies (→): If the first proposition (P) is true, then the second (Q) must also be true (e.g., P → Q).
    - Equivalent (↔): Both propositions have the same truth value (P ↔ Q).
- Example: "It is raining today (P), OR I will go for a walk (Q)." This can be written as P ∨ Q.

2. Inference:

Inference refers to the process of deriving new logical statements (conclusions) from existing ones (premises). Here are some common reasoning patterns:

- Modus Ponens: If P implies Q, and P is true, then Q is true. (P → Q, P ⊢ Q)
- Modus Tollens: If

Representation

1. Basic Elements:
    - Propositions: Statements that are either true or false.
    - Connectives: AND (∧\land∧), OR (∨\lor∨), NOT (¬\neg¬), IMPLIES (→\rightarrow→), IF AND ONLY IF (↔\leftrightarrow↔).

Inference

1. Methods:
    - Truth Tables: Enumerate all possible truth values.
    - Natural Deduction: Use inference rules (e.g., modus ponens, modus tollens).

Reasoning Patterns

1. Common Patterns:
    - Modus Ponens: If P→QP \rightarrow QP→Q and PPP is true, then QQQ is true.
    - Modus Tollens: If P→QP \rightarrow QP→Q and QQQ is false, then PPP is false.
    - Disjunctive Syllogism: If P∨QP \lor QP∨Q and ¬P\neg P¬P, then QQQ.

Resolution

1. Propositional Resolution:
    - Convert all formulas to Conjunctive Normal Form (CNF).
    - Apply resolution rule iteratively to derive a contradiction.

Syntax of Propositional Logic

The syntax of propositional logic defines the rules for creating valid propositions. In propositional logic, we combine atomic propositions using logical connectives to form more complex statements, known as compound propositions.

Building Blocks of Propositional Logic Syntax

1. Atomic Propositions:
    - These are basic statements that represent individual facts or conditions. Example:
    - P: "It is raining."
    - Q: "The heater is on."
2. Logical Connectives:
    - Connectives are used to combine atomic propositions to form compound propositions.
        - AND (∧): True if both propositions are true.
        - OR (∨): True if at least one proposition is true.
        - NOT (¬): Negates the truth value of a proposition.
        - IF-THEN (→): True unless the first proposition is true and the second is false.
        - IF AND ONLY IF (↔): True if both propositions have the same truth value.
3. Compound Propositions:
    - These are more complex statements formed by connecting atomic propositions using logical connectives. Example:
    - "If it is raining and the heater is on, then the room will be warm." This can be written in propositional logic syntax as: (P∧Q)→R

Where:

- P: "It is raining."
- Q: "The heater is on."
- R: "The room is warm."

What is Propositional Logic in Artificial Intelligence?

Propositional Logic (PL) is a branch of logic that focuses on **statements (propositions)** that can be **either true or false**. It is also known as **Boolean logic** since the truth values are binary—either **True (1)** or **False (0)**.

In AI, propositional logic forms the **foundation for logical reasoning**, allowing systems to represent **facts** and **rules** about a problem domain. These rules help the system infer new information or make decisions based on the given inputs.

Propositional logic simplifies **knowledge representation** by breaking down reasoning into **atomic statements** or **propositions**. For example, an AI system used in home automation might have propositions such as:

- **P**: "The light is on."
- **Q**: "The window is open."

Using **logical connectives**, the system can combine these propositions to represent more complex statements like:
"If the light is on and the window is open, turn off the light."

By using propositional logic, AI systems can **reason** effectively and perform tasks like **automated decision-making, knowledge representation**, and **game playing**.

Basic Facts About Propositional Logic

1. Propositions are Declarative Statements:

- In **propositional logic**, each statement, known as a **proposition**, is either **True** or **False**. Example:
- **P**: "It is raining." (True or False)
- **Q**: "The heater is on." (True or False)

2. Atomic Propositions:

- These are **simple, indivisible statements** that cannot be broken down further. Each atomic proposition represents a basic fact or condition. Example: "The door is closed."

3. Compound Propositions:

- **Multiple atomic propositions** can be combined using **logical connectives** (like AND, OR, NOT) to create **compound propositions**. Example: "The door is closed AND the heater is on."

4. Binary Truth Values:

- Every proposition has a **binary truth value**: it can only be **True (1)** or **False (0)**. There are no intermediate states. This simplicity makes propositional logic ideal for **clear-cut decisions**.

5. Logical Connectives Combine Propositions:

- Logical connectives such as **AND, OR, NOT, IF-THEN,** and **IF AND ONLY IF** allow us to create more complex propositions from simple ones.

Syntax of Propositional Logic

The **syntax** of propositional logic defines the **rules for creating valid propositions**. In propositional logic, we combine **atomic propositions** using **logical connectives** to form more complex statements, known as **compound propositions**.

Building Blocks of Propositional Logic Syntax

1. **Atomic Propositions:**
   - These are **basic statements** that represent individual facts or conditions. Example:
   - **P**: "It is raining."
   - **Q**: "The heater is on."
2. **Logical Connectives:**
   - Connectives are used to combine atomic propositions to form **compound propositions**.
     - **AND (∧)**: True if both propositions are true.
     - **OR (∨)**: True if at least one proposition is true.
     - **NOT (¬)**: Negates the truth value of a proposition.
     - **IF-THEN (→)**: True unless the first proposition is true and the second is false.
     - **IF AND ONLY IF (↔)**: True if both propositions have the same truth value.
3. **Compound Propositions:**
   - These are **more complex statements** formed by connecting atomic propositions using logical connectives. Example:
   - "If it is raining and the heater is on, then the room will be warm." This can be written in **propositional logic syntax** as: (P∧Q)→R

Where:

- **P**: "It is raining."
- **Q**: "The heater is on."
- **R**: "The room is warm."

Example of Propositional Logic

Let's explore a **real-world scenario** where propositional logic is applied in AI. Consider a **home automation system** that needs to decide whether to **turn on the air conditioner** based on the weather conditions and indoor temperature.

Scenario:

- **P**: "It is hot outside."
- **Q**: "The windows are open."
- **R**: "Turn on the air conditioner."

Using **propositional logic**, we can represent the system's decision-making with the following compound proposition:

$(P \land \neg Q) \rightarrow R$

This logic reads as:
"If it is hot outside **AND** the windows are not open, then **turn on the air conditioner**."

Explanation of the Logic:

- **AND (∧)** ensures that both conditions must be true (hot outside and windows closed) for the air conditioner to turn on.
- **NOT (¬)** negates the condition, meaning the windows must be **closed**.
- **IF-THEN (→)** states that if the first part is true, the second part (turning on the AC) will follow.

Logical Connectives in Propositional Logic

Logical connectives are essential operators that combine **atomic propositions** to form **compound propositions**. These connectives allow AI systems to build more complex rules and perform logical reasoning. Below are the most common connectives used in **propositional logic**:

Common Logical Connectives

Connective	**Symbol**	**Meaning**	**Example**
AND	∧	True if both propositions are true.	(P ∧ Q): "It is raining AND cold."
OR	∨	True if at least one proposition is true.	(P ∨ Q): "It is raining OR cold."
NOT	¬	Negates the truth value of a proposition.	¬P: "It is not raining."

IF-THEN	→	True unless the first is true and second is false.	(P → Q): "If it rains, then it will flood."
IF AND ONLY IF	↔	True if both propositions are either true or false.	(P ↔ Q): "It rains if and only if it is cloudy."

These connectives allow us to create **logical rules** that AI systems can use to make decisions. Let's take a quick look at how each works:

1. **AND (∧):**
   - The result is **True** only if both propositions are true. Example: If **P** is "It is hot" and **Q** is "The fan is on", then (P ∧ Q) means both conditions are satisfied.
2. **OR (∨):**
   - The result is **True** if at least one of the propositions is true. Example: (P ∨ Q) will be true if either it is hot or the fan is on.
3. **NOT (¬):**
   - This **inverts** the truth value of the proposition. Example: If **P** is true, ¬P will be false.
4. **IF-THEN (→):**
   - This implies that if the first proposition is true, the second must also be true for the compound statement to be true. Example: "If it rains, then the ground will be wet" (P → Q).
5. **IF AND ONLY IF (↔):**
   - This is true only when both propositions have the **same truth value** (either both true or both false). Example: "It is cloudy if and only if it will rain" (P ↔ Q).

Truth Table

A **truth table** is a useful tool for determining the **truth value** of a compound proposition based on the truth values of its atomic propositions. It systematically lists all **possible combinations** of truth values and the corresponding output for a given logical expression.

How Truth Tables Work

Let's consider two propositions:

- **P**: "It is raining."
- **Q**: "The ground is wet."

We can build a truth table to evaluate the compound proposition **P ∧ Q** (It is raining AND the ground is wet).

P	**Q**	**P ∧ Q**
TRUE	TRUE	TRUE
TRUE	FALSE	FALSE
FALSE	TRUE	FALSE
FALSE	FALSE	FALSE

- The result of **P ∧ Q** is **True** only when both **P** and **Q** are True.

Truth Table with Three Propositions

Let's extend the concept to three propositions:

- **P**: "It is hot."
- **Q**: "The air conditioner is on."
- **R**: "The windows are closed."

We can create a truth table for the compound proposition **(P ∨ Q) ∧ R** (It is hot OR the air conditioner is on, AND the windows are closed).

P	**Q**	**R**	**(P ∨ Q) ∧ R**
TRUE	TRUE	TRUE	TRUE
TRUE	TRUE	FALSE	FALSE
TRUE	FALSE	TRUE	TRUE

TRUE	FALSE	FALSE	FALSE
FALSE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	FALSE
FALSE	FALSE	TRUE	FALSE
FALSE	FALSE	FALSE	FALSE

Purpose of Truth Tables

- Truth tables help in **evaluating the outcomes** of complex logical expressions.
- They ensure **correct reasoning** by listing all possibilities, making them a vital tool for **AI systems** that rely on logical reasoning.

Precedence of Connectives in Propositional Logic

When evaluating **compound propositions** with multiple logical connectives, it's important to follow a specific **order of precedence** to ensure accurate results. Similar to arithmetic operations, **logical operators** are evaluated in a defined sequence, from highest to lowest precedence.

Order of Precedence

1. **NOT (¬)** – Negation has the **highest precedence** and is evaluated first.
2. **AND (∧)** – Conjunction is evaluated next, after negations are resolved.
3. **OR (∨)** – Disjunction comes after AND operations.
4. **IF-THEN (→)** – Implication is evaluated after OR.
5. **IF AND ONLY IF (↔)** – Biconditional has the **lowest precedence**.

Example: Precedence in Action

Consider the following logical expression:

¬P∨(Q∧R)

- **Step 1:** Evaluate **¬P** (Negation).

- **Step 2:** Evaluate **Q ∧ R** (AND).
- **Step 3:** Evaluate **¬P ∨ (Q ∧ R)** (OR).

The final result depends on the proper **evaluation order**, ensuring the correct outcome.

Using Parentheses for Clarity

To avoid ambiguity, it's good practice to use **parentheses** in complex expressions. For example:

(P∨Q)→R

In this case, **(P ∨ Q)** is evaluated first, followed by the implication →

Logical Equivalence in Propositional Logic

Logical equivalence occurs when two or more logical expressions produce the **same truth values** for all possible combinations of their propositions. In other words, two statements are logically equivalent if they always have the **same result**, regardless of the truth values of the individual propositions.

Definition of Logical Equivalence

Two propositions **P** and **Q** are logically equivalent if:

P≡Q

This means that both **P** and **Q** yield identical truth values for all possible cases. Logical equivalence allows AI systems to **simplify complex expressions** without changing their meaning.

Example of Logical Equivalence

1. **De Morgan's Laws:**
   - These laws show how **negations of conjunctions** and **disjunctions** behave:

¬(P∧Q)≡(¬P∨¬Q)

¬(P∨Q)≡(¬P∧¬Q)

2. **Double Negation:**
   - Negating a negation gives the original proposition:

¬(¬P)≡P

3. **Implication and Disjunction:**

- An implication can be rewritten as:

$P \rightarrow Q \equiv \neg P \vee Q$

Tautologies and Contradictions

- **Tautology:** A tautology is a statement that is **always true**, no matter the truth values of its individual propositions.
  - **Example:** $P \vee \neg P \equiv True$
- **Contradiction:** A contradiction is a statement that is **always false**.
  - **Example:** $P \wedge \neg P \equiv False$

Properties of Operators in Propositional Logic

In propositional logic, **logical operators** follow specific properties that allow us to **manipulate and simplify logical expressions**. Understanding these properties is essential for building efficient AI systems that rely on logical reasoning.

1. De Morgan's Laws

These laws describe how **negations** distribute over **AND (∧)** and **OR (∨)** operations:

- **First Law:**
  - $\neg(P \wedge Q) \equiv (\neg P \vee \neg Q)$

This means that the negation of a conjunction is equivalent to the disjunction of the negated propositions.

- **Second Law:**
  - $\neg(P \vee Q) \equiv (\neg P \wedge \neg Q)$

This means that the negation of a disjunction is equivalent to the conjunction of the negated propositions.

2. Commutative Property

This property states that the **order of the propositions** does not affect the result of **AND (∧)** and **OR (∨)** operations:

- **AND:**
  - $P \wedge Q \equiv Q \wedge P$
- **OR:**
  - $P \vee Q \equiv Q \vee P$

3. Associative Property

This property allows us to **group propositions** in any order when using **AND** or **OR** operations:

- **AND:**
  - (P∧Q)∧R≡P∧(Q∧R)
- **OR:**
  - (P∨Q)∨R≡P∨(Q∨R)

4. Distributive Property

This property states that **AND distributes over OR**, and vice versa:

- **AND over OR:**
  - P∧(Q∨R)≡(P∧Q)∨(P∧R)P
- **OR over AND:**
  - P∨(Q∧R)≡(P∨Q)∧(P∨R)

Applications of Propositional Logic in AI

1. Knowledge Representation in Expert Systems:

- Represents **rules and facts** to solve domain-specific problems (e.g., medical diagnosis systems).

2. Reasoning and Decision-Making:

- AI agents use logical rules to make **decisions** (e.g., robot vacuum cleaners deciding when to start cleaning).

3. Natural Language Processing (NLP):

- Helps analyze text and respond logically (e.g., chatbots understanding weather-related queries).

4. Game-Playing AI:

- Uses logic to **make strategic moves** (e.g., deciding checkmate in chess).

Limitations of Propositional Logic

1. Inability to Handle Complex Relationships

- Propositional logic cannot represent **relationships between multiple objects** or deal with hierarchies of information.

2. No Handling of Uncertainty

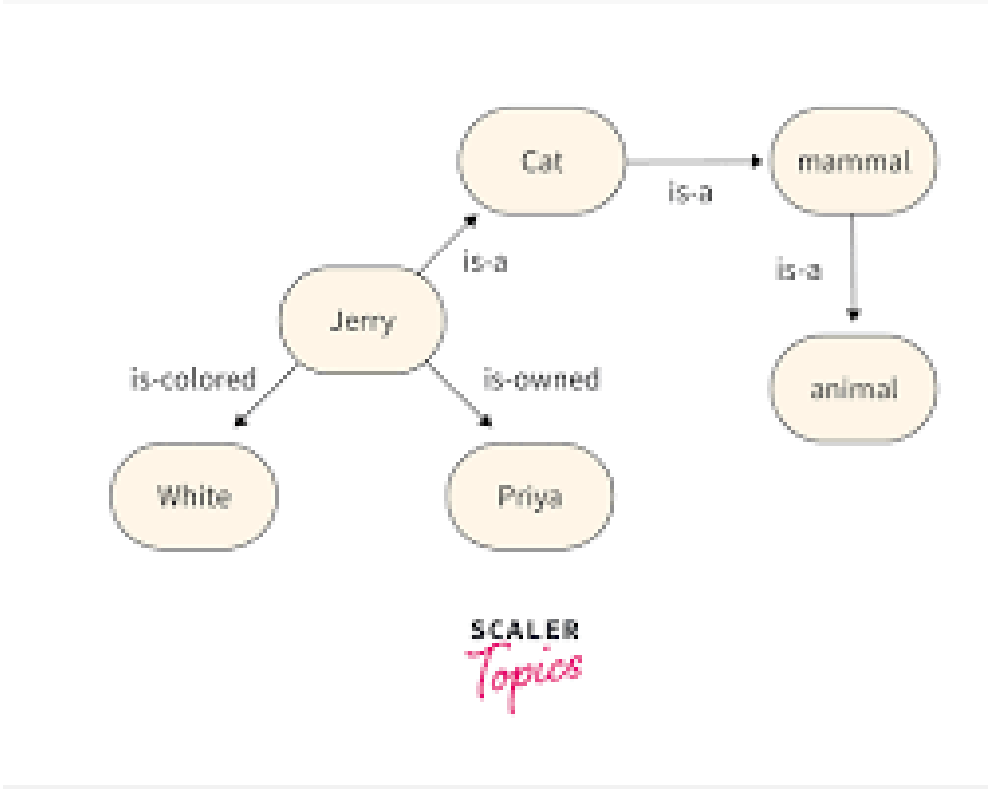
- It works only with **true or false** values and cannot deal with **probabilities or uncertain outcomes**, limiting its use in real-world applications involving incomplete data.

3. Limited Expressiveness

- It cannot represent **time-based sequences** or dynamic events, which are crucial in some AI systems like speech recognition and robotics.

4. Scalability Issues

- As the number of propositions grows, the **complexity of expressions** increases, making reasoning slower and harder to manage.



Reasoning Patterns

2. Common Patterns:
    - Modus Ponens: If P→QP \rightarrow QP→Q and PPP is true, then QQQ is true.
    - Modus Tollens: If P→QP \rightarrow QP→Q and QQQ is false, then PPP is false.
    - Disjunctive Syllogism: If P∨QP \lor QP∨Q and ¬P\neg P¬P, then QQQ.

Resolution

2. Propositional Resolution:
    - Convert all formulas to Conjunctive Normal Form (CNF).
    - Apply resolution rule iteratively to derive a contradiction.

First-order Logic

First-Order Logic (FOL), also known as predicate logic, is a knowledge representation technique used in artificial intelligence to represent objects, relationships and rules in a structured way. By extending propositional logic with predicates and quantifiers, it enables AI systems to reason about information more effectively.

- Supports logical inference and knowledge-based reasoning
- More expressive than propositional logic for complex statements
- Commonly applied in NLP, expert systems, and theorem proving

First-Order Logic

Key Components

1. Constants: These represent specific objects or entities. Example: Alice, 2, NewYork

2. Variables: These stand for unspecified objects or entities. Example: x, y, z

3. Predicates: These define properties or relationships. Example: Likes(Alice, Bob) means "Alice likes Bob"

4. Functions: It map objects to other objects. Example: MotherOf(x) refers to the mother of x

5. Quantifiers: These define the scope of variables:

- **Universal Quantifier (∀)**: Applies a predicate to all elements. Example: ∀x(Person(x)→Mortal(x))$\forall x(Person(x) \rightarrow Mortal(x))$ means "All persons are mortal"
- **Existential Quantifier (∃)**: Shows the existence of at least one element. Example: ∃x(Person(x)∧Likes(x,IceCream))$\exists x(Person(x) \wedge Likes(x, IceCream))$ means "Someone likes ice cream"

6. Logical Connectives: Include conjunction(∧∧), disjunction (∨∨), implication (→→), biconditional (↔↔) and negation (¬¬).

Syntax, Semantics and Logical Reasoning

The syntax of First-Order Logic defines the rules for constructing valid logical expressions, while semantics assigns meaning to those expressions based on a domain of interpretation. Together, they allow AI systems to represent knowledge and derive conclusions through logical reasoning.

For example, consider the following statements:

- ∀x(Cat(x)→Mammal(x))$\forall x(Cat(x) \rightarrow Mammal(x))$ means “All cats are mammals”
- ∀x(Mammal(x)→Animal(x))$\forall x(Mammal(x) \rightarrow Animal(x))$ means “All mammals are animals”
- Cat(Tom) means “Tom is a cat”

Using logical inference, we can derive:

- Mammal(Tom) meaning “Tom is a mammal”

- Animal(Tom) meaning "Tom is an animal"

This shows how First-Order Logic enables AI systems to infer new knowledge from existing facts and relationships.

Advanced Concepts

- **Unification:** Finds substitutions that make two logical expressions identical. It's used in automated reasoning to match patterns.
- **Resolution:** Uses inference rules to prove or disprove statements.
- **Model Checking:** Verifies whether a system satisfies given specifications.
- **Logic Programming:** Applies FOL in languages like Prolog for AI applications in areas like NLP and expert systems.

Propositional Logic Vs First-Order Logic

Parameter	Propositional Logic (PL)	First-Order Logic (FOL)
Representation	Represents complete statements as true or false	Represents objects, properties, and relationships
Quantifiers	Does not use quantifiers	Uses quantifiers like ∀∀ and ∃∃
Expressiveness	Limited expressiveness	Highly expressive
Reasoning Capability	Handles simple logical reasoning	Supports complex inference and reasoning
Applications	Used in simple logical systems	Used in AI, NLP, and expert systems

Advantages

- Represents complex relationships and rules more effectively than propositional logic
- Supports logical inference for deriving new knowledge from existing facts
- Uses quantifiers to express generalizations and existence statements
- Provides a structured framework for knowledge representation and reasoning

Limitations

- Computationally expensive for large knowledge bases
- Cannot handle uncertainty effectively without extensions
- Logical representation of real-world problems can become complex
- Some problems in FOL are undecidable and lack guaranteed solutions

Applications

- **Knowledge Representation:** Represents objects, properties, and relationships in a structured form for reasoning tasks

- **Automated Theorem Proving:** Applies logical rules to prove mathematical statements and verify system correctness
- **Natural Language Processing (NLP):** Helps convert natural language into logical representations for understanding and reasoning
- **Expert Systems:** Encodes domain knowledge to support decision-making in systems like medical or legal assistants
- **Semantic Web:** Defines relationships between web resources for improved search and information retrieval

Syntax and Semantics of First-Order Logic (FOL) form the foundation for representing and interpreting knowledge in artificial intelligence. Syntax defines how logical statements are correctly constructed using formal rules, while semantics explains the meaning of these statements within a specific domain or interpretation.

- Separates structure of knowledge (syntax) from meaning (semantics), making logical systems well-defined
- Provides a formal framework for building machine-understandable representations
- Forms the basis for reasoning systems that derive conclusions from given facts

Syntax

Syntax of First-Order Logic defines the formal rules for constructing valid logical expressions in AI. It specifies how symbols are arranged to form meaningful statements that represent knowledge in a structured and unambiguous way.

1. Terms

Terms represent objects or entities within the domain of discourse. In AI, terms can correspond to real-world entities, such as objects, individuals, or abstract concepts.

- **Constants:** Represent specific entities such as "John", "Apple", or "5".
- **Variables:** Act as placeholders for objects such as "x", "y".
- **Functions:** Map objects to other objects, e.g., "Age(John)", "Parent(x)".

2. Predicates

Predicates express properties, relations, or conditions that hold between objects. They describe the state of the world or assert facts about entities within the domain.

- IsHuman(x) means x is a human
- IsParent(x, y) means x is a parent of y

3. Quantifiers

Quantifiers in first-order logic allow for the specification of statements about the entirety or existence of objects within the domain.

- **Universal quantifiers (∀)**: States that a property holds for all objects, e.g., ∀xP(x)∀$xP(x)$ → "P(x) is true for every x"
- **Existential quantifiers (∃)**: Statements that hold for at least one object, e.g., ∃xP(x)∃$xP(x)$ → "There exists an x such that P(x) is true"

4. Connectives

Logical connectives are used to combine or modify simple statements into complex ones. They help express logical relationships and constraints in AI knowledge representation.

- **Conjunction (∧)**:
  - **Meaning**: Represents logical "and" between two propositions. The conjunction of two propositions is true only if both propositions are true.
  - **Example**: If P(x) represents "x is red" and Q(x) represents "x is round", then P(x)∧Q(x)$P(x)\wedge Q(x)$ represents "x is red and round".
- **Disjunction (∨)**:
  - **Meaning**: Represents logical "or" between two propositions. The disjunction of two propositions is true if at least one of the propositions is true.
  - **Example**: If P(x) represents "x is a cat" and Q(x) represents "x is a dog", then P(x)∨Q(x)$P(x)\vee Q(x)$ represents "x is either a cat or a dog".
- **Implication (→)**:
  - **Meaning**: Represents logical "if-then" relationship between two propositions. The implication P→Q$P\rightarrow Q$ is true if either Q is true or if P is false.
  - **Example**: If P(x) represents "x is a mammal" and Q(x) represents "x produces milk", then P(x)→Q(x)$P(x)\rightarrow Q(x)$ represents "if x is a mammal, then it produces milk".
- **Negation (¬)**:
  - **Meaning**: Represents logical "not" or negation of a proposition. It reverses the truth value of the proposition.
  - **Example**: If P(x) represents "x is intelligent", then ¬P(x)¬$P(x)$ represents "x is not intelligent".

Semantics

Semantics in First-Order Logic defines how logical expressions are interpreted and what they mean within a mathematical model. It connects symbolic statements with real-world meaning and determines whether a statement is true or false in a given situation.

- **Domain:** The set of all objects under consideration in a problem (such as people, numbers, or entities).
- **Interpretation:** Assigns meaning to symbols by mapping constants to objects, predicates to relations, and functions to operations in the domain.
- **Truth evaluation:** Determines whether a logical statement is true or false based on the domain and interpretation.

- **Meaning of formulas:** Explains how complete logical expressions correspond to real-world situations and how their truth value is derived.

Truth in First-Order Logic is determined by evaluating statements within a specific interpretation. A formula is considered true if it holds under the given domain and mappings of symbols.

- Universal quantification (∀∀) is true when a statement holds for every object in the domain.
- Existential quantification (∃∃) is true when a statement holds for at least one object in the domain.

Satisfaction and Validity

Satisfaction and validity describe how logical formulas are evaluated in First-Order Logic. Satisfaction refers to truth within a specific interpretation, while validity refers to truth across all interpretations.

- **Satisfaction (**M⊨ϕ*M*⊨*ϕ*)**:** A formula ϕ is satisfied by an interpretation M if it evaluates to true in that model
- **Atomic formulas:** A predicate applied to terms is satisfied if the interpretation makes the statement true
- **Quantifiers:** Universal (∀∀) applies to all objects in the domain, while Existential (∃∃) applies to at least one object
- **Validity** (⊨ϕ)(⊨*ϕ*)**:** A formula is valid if it is true in every possible interpretation
- **Relationship:** Every valid formula is satisfied in all interpretations, but a satisfied formula is not necessarily valid

Applications

- **Automated Reasoning**: Infers new facts and verifies logical or mathematical statements automatically.
- **Knowledge Representation**: Organizes domain knowledge and relationships for decision-making systems.
- **Natural Language Processing (NLP):** Converts text into logical forms and extracts structured information.
- **Planning and Problem Solving:** Generates action plans and solves constraint-based problems.
- **Robotics**: Supports robot perception, decision-making, and task planning.

Advantages

- Provides a precise and structured way to represent knowledge using formal logic
- Clearly separates syntax (structure) and semantics (meaning), improving interpretability
- Enables logical reasoning to infer new knowledge from existing statements
- Can model a wide range of real-world relationships in a consistent framework

Limitations

- Can become computationally expensive for large knowledge bases
- Struggles with uncertainty and incomplete information
- Not suitable for representing dynamic or time-dependent changes effectively
- Limited in expressing higher-order relationships beyond object-level logic

Representation

1. Syntax:
    - Variables: x,y,zx, y, zx,y,z
    - Constants: a,b,ca, b, ca,b,c
    - Functions: f(x),g(x,y)f(x), g(x,y)f(x),g(x,y)
    - Predicates: P(x),Q(x,y)P(x), Q(x,y)P(x),Q(x,y)
    - Quantifiers: Universal (∀\forall∀), Existential (∃\exists∃)

Inference

1. Methods:
    - Unification: Finding a substitution that makes different logical expressions identical.
    - Natural Deduction: Using rules of inference to derive conclusions.
    - Resolution: Extended to handle quantifiers.

Reasoning Patterns

1. Common Patterns:
    - Universal Instantiation: From ∀x P(x)\forall x \, P(x)∀xP(x), infer P(a)P(a)P(a) for any constant aaa.
    - Existential Instantiation: From ∃x P(x)\exists x \, P(x)∃xP(x), infer P(c)P(c)P(c) for some new constant ccc.

Resolution

1. First-order Resolution:
    - Process:
        - Convert formulas to prenex normal form (all quantifiers at the front).
        - Skolemize the formula (eliminate existential quantifiers by introducing Skolem functions).
        - Convert to CNF.
        - Apply resolution rule iteratively.

References

1. ***Artificial Intelligence: A New Synthesis, Harcourt Publishers*** **(Text Book)**
By N. J. Nilsson | Harcourt Publishers
2. ***Artificial Intelligence (Text Book)***
By Elaine Rich and Kevin Knight | TMH
3. ***Artificial Intelligence-Structures and Strategies For Complex Problem Solving***
By George F. Luger | Pearson Education / PHI
4. ***Artificial Intelligence-A Modern Approach***
By Stewart Russell and Peter Norvig | Pearson Education/ Prentice Hall of India | 2
5. ***Artificial Intelligence – A Practical Approach*** By Patterson | Tata McGraw Hill | 3

Parul® University
Vadodara, Gujarat

NAAC GRADE A++

https://paruluniversity.ac.in/